Exercise Sheet 05

The Scriptorium Scroll

House rules.

- One file per exercise, in `my_work/`: `ex1_split_join.py`, `ex2_write_read.py`, …
- Run everything **from the lesson folder**: `python my_work/ex3_read_csv.py`. The data lives in `data/`, and the paths on this sheet (`data/…`, `my_work/…`) only work from there.
- Type the **Given** lines first, then write the rest. Compare with the **expected output** character by character.
- Files you create go into `my_work/`, never into `data/`. The scroll must survive you.
- Hand in: commit and push to `lesson-05`, open a pull request, and tell Lars which part of the Scriptorium's copy annoyed you most.

Exercise 1 Split and join

8 min · `ex1_split_join.py`

> **RAVEN #501, from Orrin of the Clockworks:** *"The Scriptorium writes names as 'Surname, First'. And the kit list is one long string with semicolons. Don't ask. Can you take them apart?"*

GIVEN

```
name_field = "Hawkridge, Wren"
kit = "rope;torch;rations;rope;torch"
sentence = "Copying the notice board since 1019"
```

(a) Split `name_field` into surname and first name, and print "First Last".

(b) Split the sentence into words. Print how many there are, then the words in reverse order, then the words joined with underscores, all lowercase.

(c) Split the kit list, remove the duplicates (`set()`), sort them, and print them separated by ", ".

EXPECTED OUTPUT

```
Wren Hawkridge
6 words
1019 since board notice the Copying
copying_the_notice_board_since_1019
rations, rope, torch
```

Hint: `last, first = name_field.split(", ")` unpacks both halves in one go. `reversed(words)` and `sorted(set(kit_list))` each give you something `" ".join()` can eat.

Exercise 2 Write a file, read it back

10 min · `ex2_write_read.py`

> **RAVEN #502, from Queen Aldara, at dusk:** *"I keep the Scriptorium's tallies on a scrap of vellum. Biscuit ate the last one. Orrin says a 'text file' is more robust. Prove it. A."*

GIVEN

```
lines = ["Notices copied: 1019", "Notices smudged: 17", "Ink pots used: 203"]
```

(a) Write the three lines into `my_work/scriptorium_note.txt`, one per line. Use `with open(..., "w")` and remember the `"\n"`.

(b) Open the file again, read the whole thing into one string with `.read()`, print it, and print how many characters and lines it has.

(c) Open it a third time and loop over it line by line. Split each line at `": "` and print the label left-aligned in 20 characters and the number right-aligned in 6.

EXPECTED OUTPUT

```
Notices copied: 1019
Notices smudged: 17
Ink pots used: 203
Characters: 60
Lines: 3
Notices copied        1019
Notices smudged         17
Ink pots used          203
```

Hint: Each line you read still ends in `"\n"`. `.strip()` it before splitting, or the number won't line up. `print(text, end="")` stops print from adding a fourth newline.

Exercise 3 Open the scroll

8 min · `ex3_read_csv.py`

> **RAVEN #503, from Quartermistress Pell:** *"The Scriptorium's copy of the notice board is in `data/notice_board_messy.csv`. Don't open it in Excel, Excel 'helps' with the dates. Just tell me what's in it. P."*

GIVEN

```
import csv
```

(a) Open the file with `csv.DictReader` and print the column names (`reader.fieldnames`).

(b) Put all rows in a list. Print how many there are, then for the first three rows print the quest, the region and the reward, separated by " | ".

(c) Print the second row's danger and the fifteenth row's reward with `repr()`, so you can see exactly what text arrived.

EXPECTED OUTPUT

```
Columns: ['Quest', 'Title', 'Region', 'Danger', 'Reward', 'Party', 'Posted', 'Outcome']
Rows: 40
 Q0001  | THOR | 260 cr
0002 | Saltmarsh | 550 cr
Q0003 | Thornwood | 620 cr
Danger as text: '****'
Reward as text: '4,540 cr'
```

Hint: Always open CSV files with `newline=""` and `encoding="utf-8"`, or odd characters and line endings go wrong on Windows. Yes, the spaces around `Q0001` are really in the file. The Scriptorium copies by hand. The fifteenth row is `rows[14]`.

Exercise 4 Clean it up

12 min · `ex4_clean.py`

> **RAVEN #504, from Quartermistress Pell:** *"Quest numbers with and without the Q, regions cut short to four SHOUTING letters, danger as numbers or stars or '3/5', rewards with 'cr' or without. Every. Single. Moon."*

GIVEN

```
import csv

REGIONS = {"GREY": "Greyfen", "IRON": "Ironhills", "NORT": "Northern Wilds",
           "SALT": "Saltmarsh", "THOR": "Thornwood"}
```

(a) Write four functions: `clean_id(text)` (strip, uppercase, add "Q" if it's missing), `clean_region(text)` (strip, take the first four letters, uppercase, look them up in `REGIONS`), `clean_danger(text)` (stars: count them; otherwise the number before any `"/"`, as an int) and `clean_money(text)` (strip, remove "cr" and commas, then a float).

(b) Read the scroll and, for the first 8 rows, print the clean id, two spaces, the region left-aligned in 16, the danger right-aligned in 2, two spaces, and the reward as crowns right-aligned in 11.

EXPECTED OUTPUT

```
Q0001  Thornwood        1    260.00 cr
Q0002  Saltmarsh        4    550.00 cr
Q0003  Thornwood        2    620.00 cr
Q0004  Northern Wilds   2    670.00 cr
Q0005  Northern Wilds   1    410.00 cr
Q0006  Northern Wilds   2    460.00 cr
Q0007  Thornwood        2    130.00 cr
Q0008  Greyfen          4    480.00 cr
```

Hint: A dictionary from lesson 04 does the region: the first four letters of "Northern Wilds" are "Nort", and `.upper()` makes them match "NORT". `len("****")` is 4. Make the crowns text first (`f"{reward:,.2f} cr"`), then right-align it.

Exercise 5 Write the clean file

12 min · `ex5_write_csv.py`

> **RAVEN #505, from Captain Siv, quest master:** *"Good. Now can I have that as a file I can actually open? Columns: quest_id, region, danger, reward, party_size, posted, outcome. Dates the proper way round. Outcomes in one spelling. Yes?"*

GIVEN

```
import csv
```

(a) Reuse your cleaning functions (copy them in, with `REGIONS`) and add `clean_party(text)` (the first word, as an int: "4 heroes" → 4) and `clean_date(text)` (if it has a "/", it's day/month/year: turn it into year-month-day; otherwise leave it).

(b) Build a list of dicts, one per row, with exactly the seven keys Captain Siv asked for. The outcome is `.strip().title()`.

(c) Write it to `my_work/notice_board_clean.csv` with `csv.DictWriter`, header included. Print how many rows you wrote.

(d) Open your new file as plain text and print its first four lines, to prove it's real.

(e) Count the successes and print the number.

EXPECTED OUTPUT

```
Wrote 40 rows
quest_id,region,danger,reward,party_size,posted,outcome
Q0001,Thornwood,1,260.0,1,2024-01-04,Success
Q0002,Saltmarsh,4,550.0,4,2024-01-07,Success
Q0003,Thornwood,2,620.0,2,2024-01-10,Success
Successes on the board: 24
```

Hint: `day, month, year = text.split("/")`, then `f"{year}-{month}-{day}"`. `fieldnames=list(clean_rows[0])` uses the keys of the first dict as the header. Open the output with `newline=""` too, or Windows gives you a blank line after every row.

Exercise 6 Crowns per region

10 min · `ex6_region_totals.py`

> **RAVEN #506, from Queen Aldara:** *"The Privy Council asked which region earns the guild the most. I said 'The Northern Wilds, probably?' They wrote that down. Please make it true or false by the next council. A."*

GIVEN

```
import csv
```

(a) Read `data/quests.csv` (the guild ledger, 400 quests, already clean) and build two dicts: crowns paid per region and quests per region. Use the `region` and `paid` columns.

(b) Print a table, regions sorted alphabetically: name in 16, quests in 6, paid in 11 with thousands separators, average in 10 with no decimals.

(c) Print the region that paid out the most crowns.

EXPECTED OUTPUT

```
Region          Quests       Paid   Average
Greyfen             68     18,150       267
Ironhills           85     34,730       409
Northern Wilds      75     33,600       448
Saltmarsh           82     26,890       328
Thornwood           90     32,780       364
Most crowns paid out: Ironhills
```

Hint: `totals.get(region, 0) + paid` works whether the region is already in the dict or not. `max(totals, key=totals.get)` gives the key with the largest value. (So: false, Your Majesty. But only just, and the Wilds pay best per quest.)

Exercise 7 Files about files STRETCH

12 min · `ex7_pathlib_report.py`

> **RAVEN #507, from Orrin:** *"Which files are even in that data folder? And can the answer be a file too, so I can send it to the Queen by raven instead of explaining it?"*

GIVEN

```
import csv
from pathlib import Path
```

(a) Use `Path("data").glob("*.csv")` to find every CSV in the data folder. For each one (sorted), print its name left-aligned in 24 and how many data rows it has (lines minus the header) right-aligned in 5.

(b) Find the best-paid quest in the ledger (highest `paid`).

(c) Write a three-line report to `my_work/report.txt`: a title, "Best paid quest: id title (region)", and what it paid in crowns. Then print "Wrote report.txt with 3 lines" and the report itself.

EXPECTED OUTPUT

```
notice_board_messy.csv     40 rows
quests.csv                400 rows
Wrote report.txt with 3 lines
THE GUILD LEDGER: QUICK FACTS
Best paid quest: Q0231 Rid the mill of trolls at the north watchtower (Northern Wilds)
Paid: 1,800 cr
```

Hint: A `Path` can be opened like a filename, and `Path("my_work") / "report.txt"` builds a path with a slash. `report.read_text()` reads a whole file in one call. Print `report.name`, not `report`: Windows shows paths with backslashes.

Bug hunt Pages from the Grimoire, page 5

15 min · `bughunt/tally_bug.py`

Dalvric's monthly tally of the guild ledger. Quartermistress Pell says it "worked in his tower". There are **three** bugs: one crashes before a single row is read, one gives a wrong count without crashing, and one loses the pennies.

EXPECTED OUTPUT ONCE FIXED

```
Quests: 400
Deadly (difficulty 5): 40
Total paid: 146,150 cr
Average paid: 365.38 cr
```

Commit each fix separately, with a message explaining what was wrong.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.